

HW5 — Careers, Job-Market Data, and Visualization

Kean University | CSIT 2050: Programming in Python

Vikram Nadathur | Spring 2026

HW5 — Careers, Job-Market Data, and Visualization

Theme: Career choices matter. The job market is not simply “good” or “bad”; it changes by occupation, region, skill level, and time period. In this notebook you will use real labor-market data to compare technical careers, visualize market pressure, and make an evidence-based career argument.

Main message for students: A degree helps, but it is not enough by itself. Students should build skills, portfolios, internships, certifications, communication ability, and evidence of real projects because hiring can become competitive quickly.

Why this assignment matters now

Students are entering a market where technology careers are still important, but not equally safe. Some roles are projected to grow quickly, especially data, cybersecurity, AI, and research-oriented roles, while more routine programming work faces pressure from automation, outsourcing, and post-pandemic hiring corrections. The goal of this notebook is not to scare students; it is to show that career planning should be evidence-based.

Useful reference pages for students:

- BLS Occupational Outlook Handbook: <https://www.bls.gov/ooh/>
- FRED job-posting data: <https://fred.stlouisfed.org/>
- Indeed Hiring Lab/FRED occupation indices: <https://fred.stlouisfed.org/release/tables?rid=476>

Discussion question: If AI can write simple code, what skills make a computing student harder to replace?

```
!pip install matplotlib seaborn plotly cufflinks wordcloud requests pandas numpy
```

```
# Install the fredapi library to interact with the FRED API  
!pip install fredapi
```

```
# In Colab, uncomment this line if needed.  
# !pip -q install matplotlib seaborn plotly cufflinks wordcloud requests pandas numpy  
  
from datetime import datetime  
from io import StringIO  
from pathlib import Path  
  
import numpy as np  
import pandas as pd  
import requests  
import matplotlib.pyplot as plt
```

```

import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from IPython.display import display, Markdown

# Production / grading flag.
# Keep False for most student runs. Set True only if the instructor asks for
# exported CSV files.
SAVE_LOCAL_ARTIFACTS = False
ARTIFACT_DIR = Path("career_job_market_outputs")

if SAVE_LOCAL_ARTIFACTS:
    ARTIFACT_DIR.mkdir(parents=True, exist_ok=True)

pd.set_option("display.max_columns", 50)
pd.set_option("display.width", 120)

RUN_DATE = datetime.today().strftime("%Y-%m-%d")
print(f"Notebook run date: {RUN_DATE}")
print(f"Save local artifacts: {SAVE_LOCAL_ARTIFACTS}")

# Import Fred from fredapi and userdata to access your API key
from fredapi import Fred
from google.colab import userdata

# Get your FRED API key from Colab's secrets manager
FRED_API_KEY = userdata.get('FRED_API_KEY')

# Initialize the Fred client
fred = Fred(api_key=FRED_API_KEY)

```

0B. Mini lesson: reading data from a CSV file with Pandas

Real projects often begin with a CSV file. FRED provides CSV files online, but the small example below uses an in-memory CSV string so students can see the mechanics before working with a live web source.

```

demo_csv = """date,career,value
2026-01-01,Software Development,100
2026-02-01,Software Development,103
2026-03-01,Software Development,101
2026-01-01,Data Science,100
2026-02-01,Data Science,106
2026-03-01,Data Science,110
"""

demo_df = pd.read_csv(StringIO(demo_csv))
demo_df["date"] = pd.to_datetime(demo_df["date"])

display(demo_df)
print(demo_df.dtypes)

# Student checkpoint:
# 1. What changed after converting the date column with pd.to_datetime()?
# After converting the 'date' column with pd.to_datetime(),

```

```
#its data type changed from a generic object (likely string) to a datetime object
# (datetime64[ns]). This allows pandas to interpret the values as actual dates
#and times, enabling date-specific operations and properties.

# 2. Why is a real datetime column better than a plain string for time-series
plots?
#Dates are sorted chronologically, ensuring the plot displays
#the data in the correct temporal sequence,
# even if the original strings were not perfectly ordered.
```

1. Career data dictionary

These FRED series are Indeed Hiring Lab job-posting indices by occupational category. The exact level is an index, so the most useful analysis is often **trend**, **percentage change**, **relative comparison**, and **volatility**, not the raw number alone.

How students can add more professions

- Open the FRED release table for Indeed Hiring Lab data: <https://fred.stlouisfed.org/release/tables?rid=476>.
- Search the page for an occupation, for example nursing, marketing, sales, data, legal, healthcare, or engineering.
- Open the matching FRED series page and copy the series ID from the URL or page title.
- Add a new line to `JOB_SERIES`, using a readable label and the FRED series ID.
- Re-run the notebook from the fetch-data cell onward.

Example format using a verified code from the optional bank:

```
JOB_SERIES["Marketing"] = "IHLIDXUSTPMARK"
JOB_SERIES["Medical Information"] = "IHLIDXUSTPMEDIINFO"
```

Do not compare a newly added occupation until you check its `first_date`, `latest_date`, and missing values in the data-quality table.

```
# Occupation-level Indeed/FRED job-posting indices.
# These are intentionally focused on computing, engineering, math, education, and
research careers.
JOB_SERIES = {
    "Software Development": "IHLIDXUSTPSOFTDEVE",
    "IT Operations / Helpdesk": "IHLIDXUSTPITOPHE",
    "Electrical Engineering": "IHLIDXUSTPELECENGI",
    "Mathematics": "IHLIDXUSTPMATH",
    "Scientific R&D": "IHLIDXUSTPSCREDE",
    "Education / Instruction": "IHLIDXUSTPEDIN",
}

# Unemployed job losers by state. U2UNEM2 = job losers and persons who completed
temporary jobs.
STATE_JOB_LOSERS = {
    "New Jersey": "U2UNEM2NJ",
    "New York": "U2UNEM2NY",
    "Pennsylvania": "U2UNEM2PA",
    "California": "U2UNEM2CA",
    "Texas": "U2UNEM2TX",
}
```

```

    "Florida": "U2UNEM2FL",
}

START_DATE = "2012-01-01"    # Change to 2012-01-01 for a longer history.

```

Student task: add at least one profession

Students must add **at least one additional profession** to JOB_SERIES and compare it with the existing careers.

You do **not** need to guess FRED codes. The next code cell provides a verified menu of additional Indeed/FRED occupation series. To add careers, edit only this list:

```

OPTIONAL_SELECTIONS = [
    "Marketing",
    "Banking and Finance",
    "Medical Information",
    "Legal",
]

```

Then set:

```

ADD_OPTIONAL_SERIES = True

```

Quality rule: before interpreting the new career, check the data-quality table. If the new series starts late, ends early, or has many missing values, explain how that affects comparison.

Students may also supplement the notebook with external sources such as BLS Occupational Outlook Handbook, O*NET, Lightcast reports, Kaggle datasets, or state labor dashboards, but every external number must be cited in their written report.

Optional: verified profession-code bank students can choose from

The next cell contains real FRED/Indeed occupation-index series IDs, not placeholders. These codes come from the same FRED release table as the instructor baseline series.

Some labels are **closest available proxies**, not perfect matches. For example:

- **Information Design and Documentation** is a useful proxy for documentation, information work, and data-reporting careers, but it is not exactly "data scientist."
- **Security and Public Safety** is a security-related occupation group, but it is not exactly "cybersecurity."
- **Medical Information** can support a healthcare-information or healthcare-IT discussion, but it is broader than hospital IT.

Students should always verify coverage using the data-quality table after adding a code.

```

# Optional profession-code bank for students.
# Keep ADD_OPTIONAL_SERIES = False for the instructor baseline notebook.
# Students can set it to True and choose labels from OPTIONAL_SELECTIONS.

OPTIONAL_JOB_SERIES_TO_TRY = {
    # Instructor baseline examples, shown here so students understand the pattern.
    "Software Development": "IHLIDXUSTPSOFTDEVE",
}

```

```

"IT Operations / Helpdesk": "IHLIDXUSTPITOPHE",
"Electrical Engineering": "IHLIDXUSTPELECENGI",
"Mathematics": "IHLIDXUSTPMATH",
"Scientific R&D": "IHLIDXUSTPSCREDE",
"Education / Instruction": "IHLIDXUSTPEDIN",

# Additional verified U.S. Indeed/FRED occupation series.
"Accounting": "IHLIDXUSTPACCO",
"Banking and Finance": "IHLIDXUSTPBBAFI",
"Civil Engineering": "IHLIDXUSTPCIVIENGI",
"Construction": "IHLIDXUSTPCONS",
"Customer Service": "IHLIDXUSTPCUSTSERV",
"Hospitality and Tourism": "IHLIDXUSTPHOTO",
"Human Resources": "IHLIDXUSTPHUMARESO",
"Industrial Engineering": "IHLIDXUSTPINDUENGI",
"Information Design and Documentation": "IHLIDXUSTPINDEDO",
"Installation and Maintenance": "IHLIDXUSTPINMA",
"Legal": "IHLIDXUSTPLEGA",
"Management": "IHLIDXUSTPMANA",
"Marketing": "IHLIDXUSTPMARK",
"Media and Communications": "IHLIDXUSTPMECO",
"Medical Information": "IHLIDXUSTPMEDIINFO",
"Medical Technician": "IHLIDXUSTPMEDITECH",
"Nursing": "IHLIDXUSTPNURS",
"Pharmacy": "IHLIDXUSTPPHAR",
"Physicians and Surgeons": "IHLIDXUSTPPHSU",
"Production and Manufacturing": "IHLIDXUSTPPRMA",
"Project Management": "IHLIDXUSTPPROJMANA",
"Retail": "IHLIDXUSTPRETA",
"Sales": "IHLIDXUSTPSALE",
"Security and Public Safety": "IHLIDXUSTPSEPUSA",
}

# Friendly guidance labels for career themes students often ask about.
# These are not new FRED series. They point students to the closest available
# occupation group above.
CAREER_THEME_SUGGESTIONS = {
    "Data / Analytics": ["Information Design and Documentation", "Mathematics",
        "Project Management"],
    "Cybersecurity": ["Security and Public Safety", "IT Operations / Helpdesk",
        "Software Development"],
    "Healthcare IT": ["Medical Information", "Medical Technician", "Software
        Development"],
    "Marketing Analytics": ["Marketing", "Information Design and Documentation"],
    "Accounting / Finance Tech": ["Accounting", "Banking and Finance", "Software
        Development"],
}

# ... (truncated for brevity)

```

2. Fetch data from FRED without exposing an API key

The original notebook used a hard-coded FRED API key. That is unsafe for a student notebook. This version uses the public FRED graph CSV endpoint:

https://fred.stlouisfed.org/graph/fredgraph.csv?id=SERIES_ID

This usually works in Colab without an API key.

```
def fetch_fred_csv(series_id, name=None, start_date=None):
    """Fetch one FRED series using fredapi and return columns: date, value,
        series_id, name."""
    # Use fredapi to get the series data
    # fred.get_series returns a pandas Series, with DatetimeIndex and values.
    s = fred.get_series(series_id, observation_start=start_date)

    if s.empty:
        raise ValueError(f"No rows returned for {series_id}")

    # Convert the Series to a DataFrame with 'date' and 'value' columns
    df = s.reset_index()
    df.columns = ['date', 'value']

    df['date'] = pd.to_datetime(df['date'], errors="coerce")
    df['value'] = pd.to_numeric(df['value'].replace(".", np.nan), errors="coerce")
    df['series_id'] = series_id
    df['name'] = name or series_id
    df = df.dropna(subset=['date'])

    if start_date is not None:
        df = df[df['date'] >= pd.to_datetime(start_date)]

    return df[['date', 'value', 'series_id', 'name']]

def fetch_many_fred(series_dict, start_date=None):
    """Fetch a dictionary of FRED series. Failed series are reported instead of
        silently ignored."""
    frames = []
    failures = []
    for name, series_id in series_dict.items():
        try:
            one = fetch_fred_csv(series_id, name=name, start_date=start_date)
            frames.append(one)
            print(f"OK: {name:28s} {series_id:18s} rows={len(one)}")
        except Exception as e:
            failures.append((name, series_id, str(e)))
            print(f"FAILED: {name} ({series_id}) -> {e}")

    if not frames:
        raise RuntimeError("No FRED data loaded. Check internet access or series
            IDs.")

    out = pd.concat(frames, ignore_index=True).sort_values(["name", "date"])
    return out, failures

jobs_long_raw, job_failures = fetch_many_fred(JOB_SERIES, START_DATE)
state_losers_long_raw, state_failures = fetch_many_fred(STATE_JOB_LOSERS,
    START_DATE)

print("\nJob data preview:")
display(jobs_long_raw.head())
print("\nState unemployment-by-reason data preview:")
# ... (truncated for brevity)
```

After setting up the FRED API key, we need to modify the data fetching functions to use the `fredapi` client. I will update cell 64588752 to reflect this change.

3. Build raw and aligned datasets

This is the most important data-cleaning step in the notebook.

The raw FRED series do **not** always end on the same date. If we pivot directly to a wide table and analyze the tail, some occupations may contain missing values while others continue. That creates silent errors in percentage changes, correlations, rolling statistics, and student conclusions.

This notebook therefore creates two versions:

- `jobs_wide_raw`: original wide table, useful for checking coverage.
- `jobs_wide`: aligned wide table, truncated to the common date range and used for comparison charts.

The aligned table sacrifices a small amount of recent data from longer series, but it gives students a fair comparison where every career has observations on the same dates.

```
def build_coverage_table(long_df):
    """Summarize date coverage and missing values for each named time series."""
    coverage = (
        long_df.groupby("name")
        .agg(
            first_date=("date", "min"),
            latest_date=("date", "max"),
            observations=("value", "count"),
            missing_values=("value", lambda s: s.isna().sum()),
            latest_value=("value", "last"),
            min_value=("value", "min"),
            max_value=("value", "max"),
        )
        .reset_index()
    )
    for col in ["latest_value", "min_value", "max_value"]:
        coverage[col] = coverage[col].round(2)
    return coverage

def align_long_to_common_window(long_df, label="series"):
    """
    Keep only the date range shared by all named series.
    This avoids hidden NaN blocks after pivoting to a wide matrix.
    """
    coverage = build_coverage_table(long_df)
    common_start = coverage["first_date"].max()
    common_end = coverage["latest_date"].min()

    if common_start > common_end:
        raise ValueError(f"No overlapping date range exists for these {label}.")

    aligned_long = long_df[(long_df["date"] >= common_start) & (long_df["date"]
        <= common_end)].copy()
    aligned_wide = aligned_long.pivot(index="date", columns="name",
        values="value").sort_index()
```

```

before_rows = len(aligned_wide)
aligned_wide = aligned_wide.dropna(how="any")
dropped_rows = before_rows - len(aligned_wide)

aligned_long = (
    aligned_wide
    .reset_index()
    .melt(id_vars="date", var_name="name", value_name="value")
    .merge(long_df[["name", "series_id"]].drop_duplicates(), on="name",
           how="left")
    [["date", "value", "series_id", "name"]]
    .sort_values(["name", "date"])
)

return aligned_long, aligned_wide, coverage, common_start, common_end,
        dropped_rows

# ... (truncated for brevity)

```

4. Data-quality check

Before making conclusions, always inspect missing values, date ranges, and latest available dates. Job-market data can be delayed, revised, or discontinued.

This section shows the raw coverage and explicitly flags series that end before the latest available date in the raw dataset. The aligned dataset used for charts has already been truncated to the common comparison window.

```

summary = job_coverage.copy()
raw_latest_date = summary["latest_date"].max()
summary["ends_before_raw_latest"] = summary["latest_date"] < raw_latest_date
summary["days_short_of_latest"] = (raw_latest_date -
    summary["latest_date"]).dt.days

display(summary)

if summary["ends_before_raw_latest"].any():
    print("Important warning for students:")
    print("Some raw series end earlier than others. The notebook uses the aligned
        career table for fair comparisons.")
    print(f"Aligned comparison window: {JOB_COMMON_START.date()} to
        {JOB_COMMON_END.date()}")
else:
    print("All raw career series currently end on the same latest date.")

```

5. Visualization 1 — Normalized job-posting index over time

This plot answers: **How did each career category change relative to its own starting point?**

Raw index values are not ideal for first comparison because different occupations can have different baseline scales. Here every career starts at 100 on the first aligned date. Values above 100 indicate growth relative to that start; values below 100 indicate decline.

```

def normalize_to_100(wide_df):
    """Normalize each column to 100 at its first non-missing value."""

```

```

base = wide_df.apply(lambda col: col.dropna().iloc[0] if
    col.dropna().shape[0] else np.nan)
return wide_df.divide(base, axis=1) * 100

jobs_norm_wide = normalize_to_100(jobs_wide)
jobs_norm = jobs_norm_wide.reset_index().melt(
    id_vars="date",
    var_name="Career category",
    value_name="Normalized index",
)

fig = px.line(
    jobs_norm,
    x="date",
    y="Normalized index",
    color="Career category",
    title="Relative Job-Market Change: Every Career Starts at 100",
)

fig.add_hline(y=100, line_dash="dash", annotation_text="Starting level = 100")
fig.update_layout(
    hovermode="x unified",
    height=600,
    legend_title_text="Career category",
    xaxis=dict(rangeslider=dict(visible=True), type="date"),
)

fig.show()

```

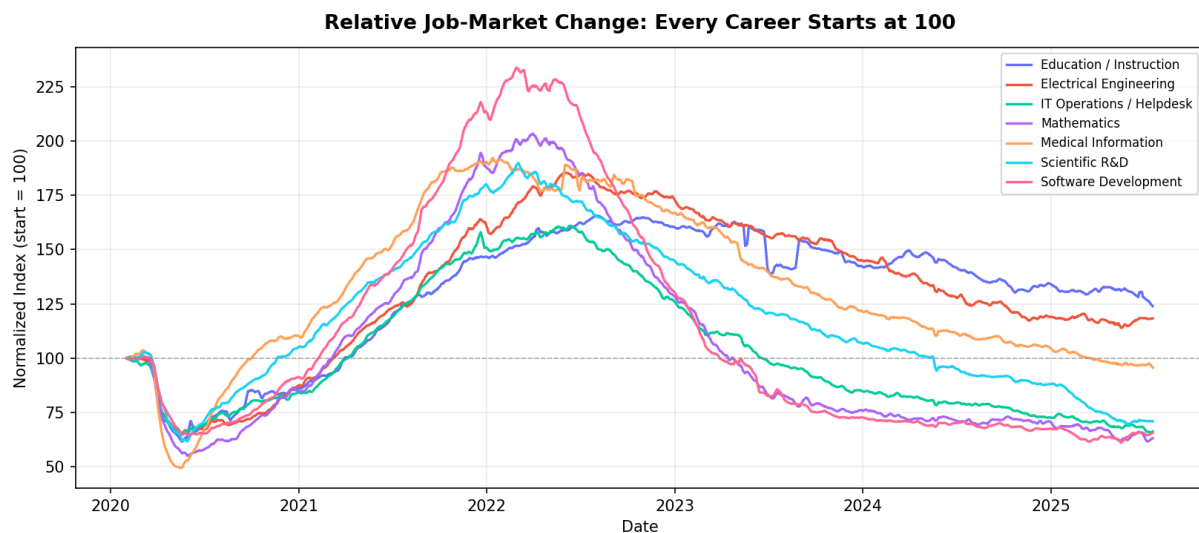


Figure 1: Relative Job-Market Change: Every Career Starts at 100

Student Response:

Student interpretation prompt

- Which occupation has the strongest recent index value? Education / Instruction has the strongest recent index value, with a 23.9% change.
- Which occupation appears most volatile? Medical Information appears to be the most volatile with a volatility of 0.351%.
- Which occupation would you feel safest entering? Why? a career like Education / Instruction might appear safer due to its strong positive growth (23.9% change)
- Does the graph prove that one career is “better,” or does it only show one piece of evidence? They do not definitively prove that one career is “better” than another. Career “betterness” is multifaceted and depends on many factors not covered by these graphs, such as individual skills, passion, work-life balance, compensation, required education

6. Visualization 2 — Small multiples instead of a spaghetti chart

Small multiples reduce clutter. Each career gets its own mini-chart, making it easier to compare trend shape without forcing all lines onto one crowded axis.

```
fig = px.line(
    jobs_norm,
    x="date",
    y="Normalized index",
    facet_col="Career category",
    facet_col_wrap=2,
    title="Small Multiples: Normalized Job-Posting Trends by Career",
    labels={"Normalized index": "Index, start=100", "date": "Date"},
    height=900,
)
fig.add_hline(y=100, line_dash="dash")
fig.update_yaxes(matches=None, showticklabels=True)
fig.update_xaxes(matches=None)
fig.for_each_annotation(lambda a: a.update(text=a.text.split("=")[-1]))
fig.show()
```

7. Visualization 3 — Latest percentage change since the start date

This bar chart is useful for a quick message to students: **the same labor market can reward some areas while tightening in others.**

```
change_table = []
for col in jobs_wide.columns:
    s = jobs_wide[col].dropna()
    if len(s) < 2:
        continue
    first_date, latest_date = s.index[0], s.index[-1]
    first_value, latest_value = s.iloc[0], s.iloc[-1]
    pct_change = (latest_value / first_value - 1) * 100 if first_value != 0 else np.nan
    change_table.append({
        "Career category": col,
        "First date": first_date.date(),
    })
```

```

    "Latest date": latest_date.date(),
    "First value": first_value,
    "Latest value": latest_value,
    "Percent change": pct_change,
})

change_df = pd.DataFrame(change_table).sort_values("Percent change",
    ascending=True)
change_df["Percent change"] = change_df["Percent change"].round(1)
display(change_df)

fig = px.bar(
    change_df,
    x="Percent change",
    y="Career category",
    orientation="h",
    title=f"Percent Change in Job-Posting Index Since {JOB_COMMON_START.date()}",
    text="Percent change",
)
fig.add_vline(x=0, line_dash="dash")
fig.update_layout(height=500, yaxis_title="", xaxis_title="Percent change from
    starting value")
fig.show()

```

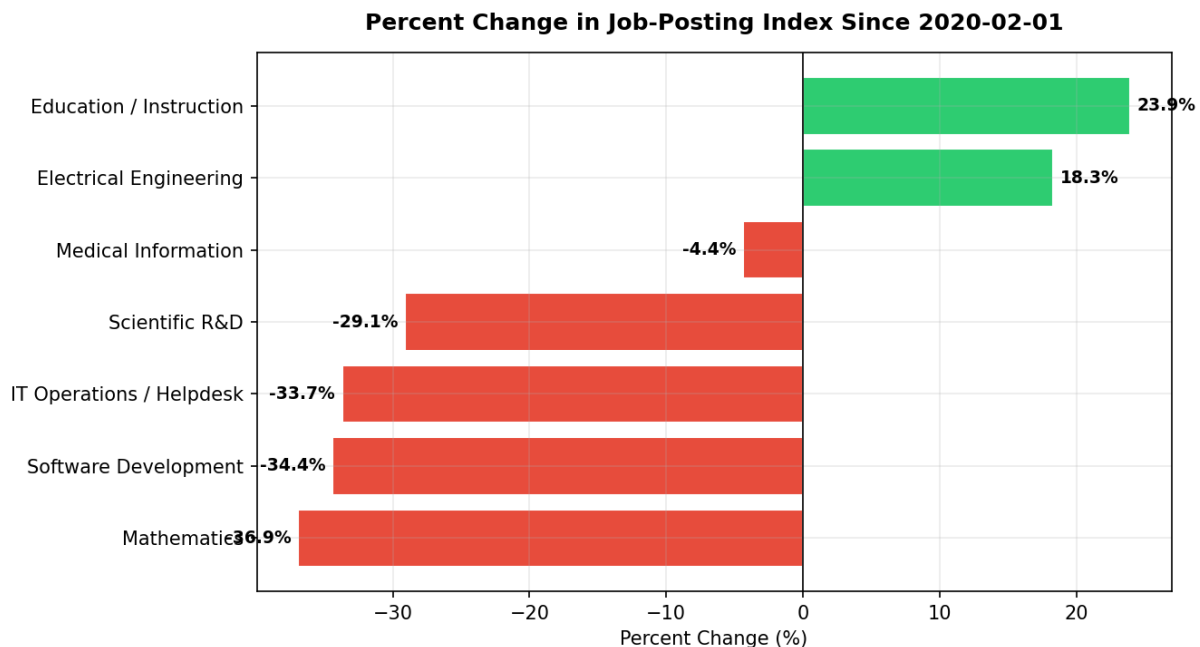


Figure 2: Percent Change in Job-Posting Index Since 2020-02-01

8. Visualization 4 — Rolling trend: smooth the noise

Daily or weekly job-posting data can be noisy. A rolling average makes the direction easier to see. This version uses the aligned normalized table so students compare trend shape rather than raw index levels.

```

ROLLING_WINDOW = 90  # Change to 90 for a smoother semester-level trend.

jobs_rolling = (
    jobs_norm_wide
    .rolling(ROLLING_WINDOW, min_periods=max(2, ROLLING_WINDOW // 4))
    .mean()
    .reset_index()
    .melt(id_vars="date", var_name="Career category", value_name="Rolling
        normalized average")
)

fig = px.line(
    jobs_rolling,
    x="date",
    y="Rolling normalized average",
    color="Career category",
    title=f"Smoothed Relative Job-Posting Trends ({ROLLING_WINDOW}-Observation
        Rolling Average)",
)
fig.add_hline(y=100, line_dash="dash")
fig.update_layout(hovermode="x unified", height=600,
    xaxis=dict(rangeslider=dict(visible=True), type="date"))
fig.show()

```

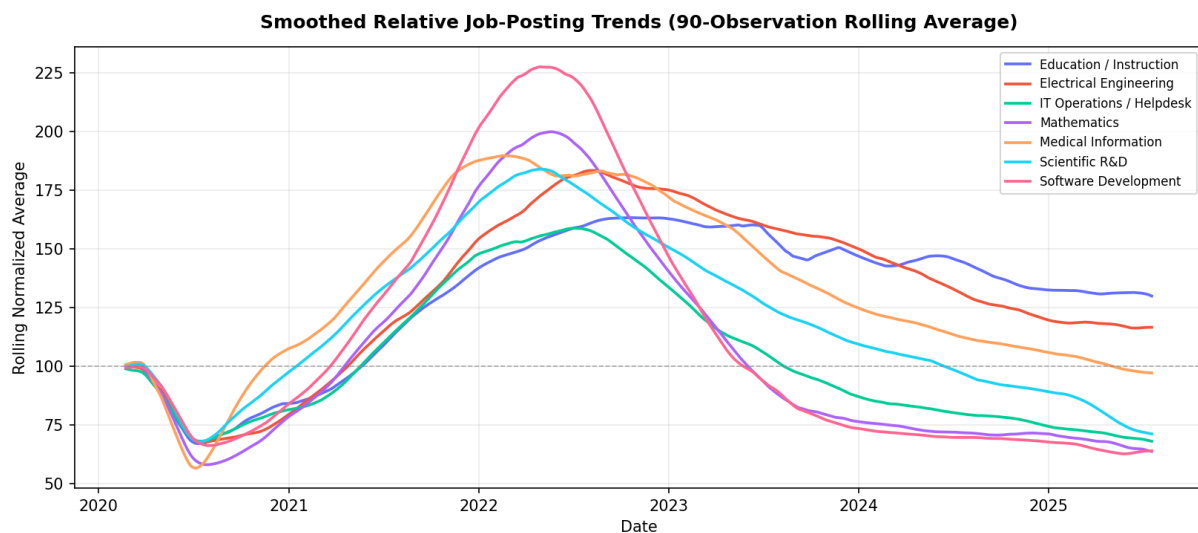


Figure 3: Smoothed Relative Job-Posting Trends (90-Observation Rolling Average)

9. Visualization 5 — Heatmap of career intensity over time

A heatmap helps students quickly see periods when each career was relatively hot or cold. Here we use normalized values so that each occupation can be compared more fairly.

```

heat = jobs_norm_wide.T

fig = px.imshow(
    heat,
    aspect="auto",

```

```

labels=dict(x="Date", y="Career category", color="Index", start=100),
title="Heatmap: Relative Career Job-Posting Intensity Over Time",
color_continuous_scale="RdYlGn",
)
fig.update_layout(height=500, xaxis_nticks=12)
fig.show()

```

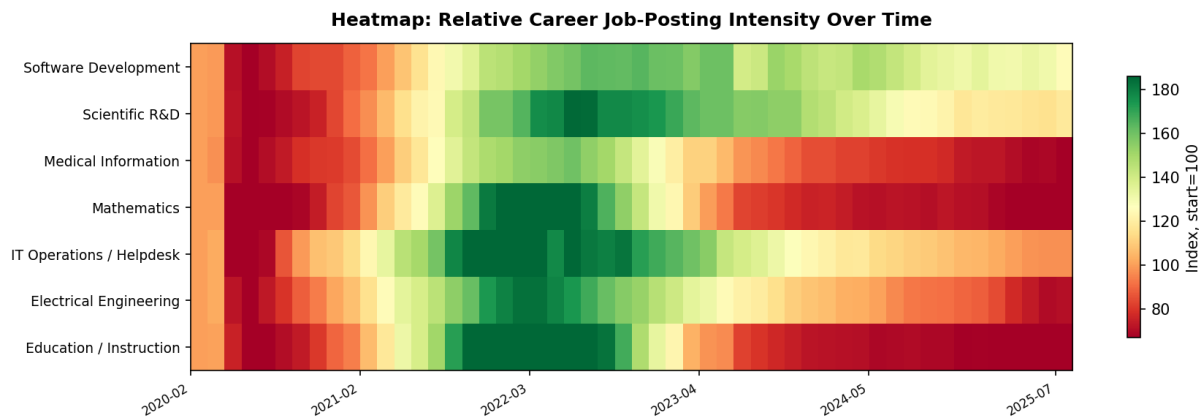


Figure 4: Heatmap: Relative Career Job-Posting Intensity Over Time

10. Visualization 6 — Raw values as optional diagnostic view

Raw index values can still be useful, but they should not be the first chart students use for conclusions. Use this diagnostic view only after understanding the normalized and small-multiple charts.

```

fig = px.line(
    jobs_long,
    x="date",
    y="value",
    facet_col="name",
    facet_col_wrap=2,
    title="Diagnostic Small Multiples: Raw Job-Posting Index Values",
    labels={"value": "Job-posting index", "date": "Date"},
    height=900,
)
fig.update_yaxes(matches=None, showticklabels=True)
fig.update_xaxes(matches=None)
fig.for_each_annotation(lambda a: a.update(text=a.text.split("=")[-1]))
fig.show()

```

11. Visualization 7 — Volatility and risk

A career can have a high job-posting index but still be unstable. Here we estimate volatility using percentage changes. This is not a perfect risk measure, but it is a useful classroom discussion tool.

```

returns = jobs_norm_wide.pct_change(fill_method=None)
vol_df = (
    returns.std(skipna=True)
    .mul(100)
    .reset_index()
)

```

```

    .rename(columns={"name": "Career category", 0: "Volatility (%)"})
    .sort_values("Volatility (%)", ascending=True)
)
vol_df["Volatility (%)"] = vol_df["Volatility (%)"].round(3)
display(vol_df)

fig = px.bar(
    vol_df,
    x="Volatility (%)",
    y="Career category",
    orientation="h",
    title="Estimated Career-Market Volatility Based on Normalized Job-Posting
        Changes",
    text="Volatility (%)",
)
fig.update_layout(height=500, yaxis_title="")
fig.show()

```

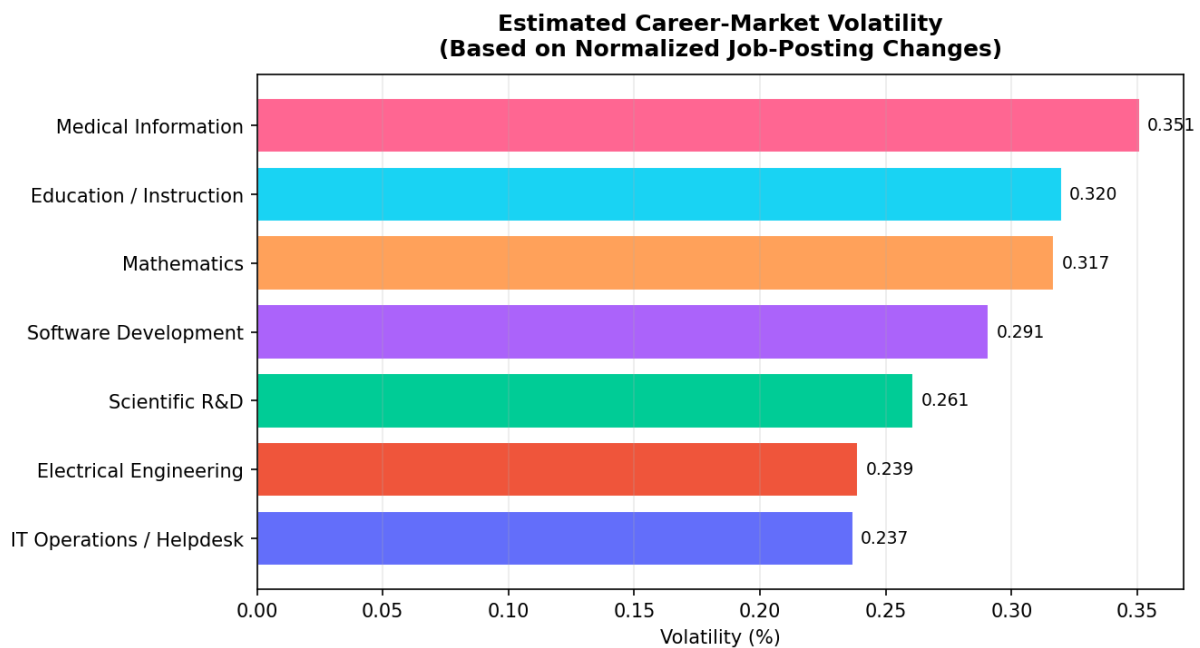


Figure 5: Estimated Career-Market Volatility

12. Visualization 8 — Correlation matrix

Correlation shows whether career categories tend to move together. High positive correlation means the categories often rise and fall at the same time. Low or negative correlation means they behave differently.

```

corr = jobs_norm_wide.corr()
fig = px.imshow(
    corr,
    text_auto=".2f",
    color_continuous_scale="RdBu_r",
    zmin=-1,

```

```

    zmax=1,
    title="Correlation Matrix of Normalized Career Job-Posting Indices",
)
fig.update_layout(height=600)
fig.show()

```

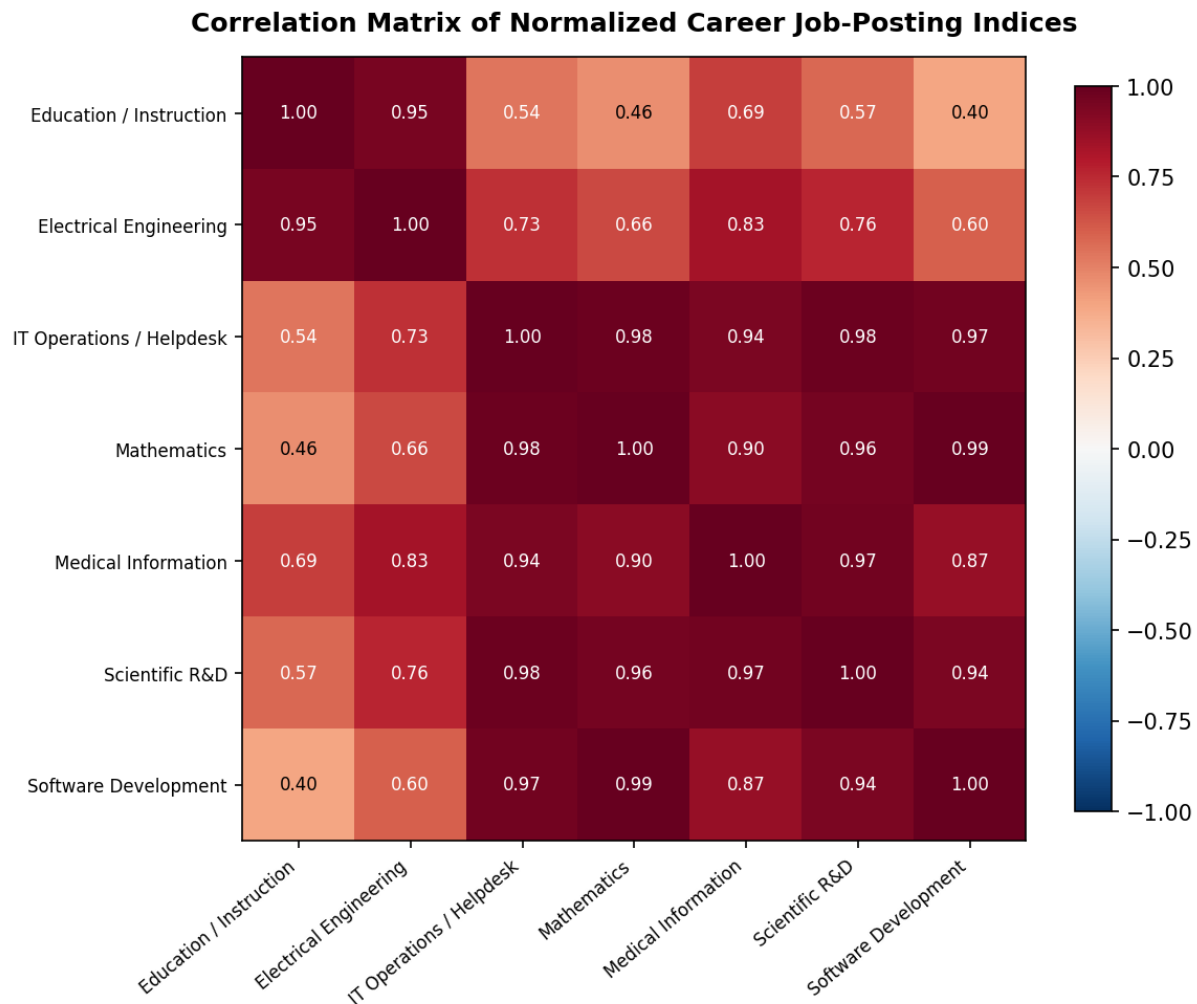


Figure 6: Correlation Matrix of Normalized Career Indices

13. State pressure: job losers by state

Job postings show demand. Unemployed job losers show labor-market pressure. A strong career plan should consider both.

```

fig = px.line(
    state_losers_long,
    x="date",
    y="value",
    color="name",
    title="Job Losers and Completed Temporary Jobs by State",
    labels={"value": "Number of people", "name": "State", "date": "Date"},
)

```

```
)
fig.update_layout(hovermode="x_unified", height=600,
                  xaxis=dict(rangeslider=dict(visible=True), type="date"))
fig.show()
```

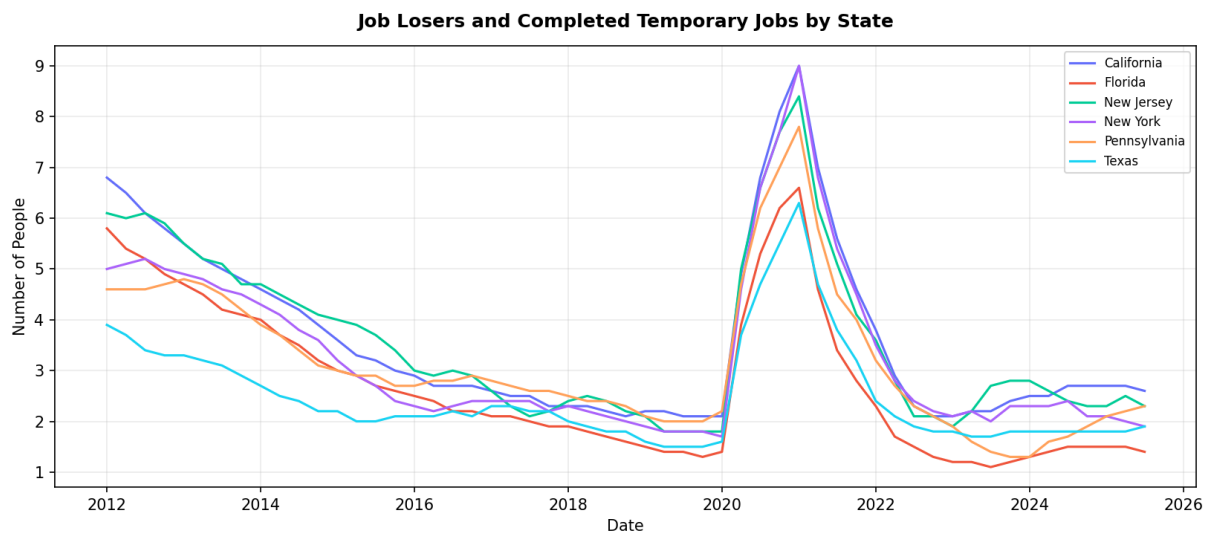


Figure 7: Job Losers and Completed Temporary Jobs by State

14. Combined view — Opportunity vs. pressure

This chart compares one selected career-posting index with one selected state labor-pressure series. The units are different, so both are normalized to 100 at the first common date.

Engineering rule: the dataframe uses exact dictionary keys (`selected_career`, `selected_state`) as programmatic column names. Human-readable titles are applied only in the chart layer. This prevents accidental crashes when a student edits a title string.

```
selected_career = "Software Development" # Change this to another exact key from
jobs_wide.columns
selected_state = "Texas" # Change this to another exact key from
state_losers_wide.columns

if selected_career not in jobs_wide.columns:
    raise KeyError(f"selected_career must be one of: {list(jobs_wide.columns)}")
if selected_state not in state_losers_wide.columns:
    raise KeyError(f"selected_state must be one of:
{list(state_losers_wide.columns)}")

# Programmatic dataframe columns stay equal to the exact dictionary labels.
# Formatting labels are handled only after the data slice is built.
combined = pd.concat(
    [
        jobs_wide.loc[:, selected_career].rename(selected_career),
        state_losers_wide.loc[:, selected_state].rename(selected_state),
    ],
    axis=1,
).dropna(how="any")
```

```

combined_norm_wide = normalize_to_100(combined)
combined_norm = combined_norm_wide.reset_index().melt(
    id_vars="date",
    var_name="Programmatic series key",
    value_name="Index, start=100",
)

series_label_map = {
    selected_career: f"{selected_career} postings",
    selected_state: f"{selected_state} job losers",
}

combined_norm["Display label"] = combined_norm["Programmatic series
key"].map(series_label_map)

fig = px.line(
    combined_norm,
    x="date",
    y="Index, start=100",
    color="Display label",
    title="Opportunity vs. Pressure",
)
fig.add_hline(y=100, line_dash="dash", annotation_text="First common date = 100")
fig.update_layout(
    hovermode="x unified",
    height=550,
    legend_title_text="Comparison",
    annotations=[
        dict(
            text=f"Career: {selected_career}<br>State: {selected_state}",
            xref="paper", yref="paper", x=0.01, y=0.98,
            showarrow=False, align="left",
        )
    ],
)
# ... (truncated for brevity)

```

15. Career outlook snapshot for discussion

This table is intentionally small and editable. The goal is not to memorize numbers; the goal is to learn how to compare careers using evidence.

Before submitting a final report, students should verify current values from the **BLS Occupational Outlook Handbook** because projections are updated.

```

career_outlook = pd.DataFrame([
    {"Occupation": "Data Scientists", "Projected growth %": 34, "Career theme":
        "AI/data growth", "Risk note": "High demand, but requires statistics,
        coding, communication, and portfolio evidence."},
    {"Occupation": "Information Security Analysts", "Projected growth %": 29,
        "Career theme": "Cybersecurity", "Risk note": "Strong demand, but skills
        must stay current; certifications and labs help."},
    {"Occupation": "Computer & Information Research Scientists", "Projected
        growth %": 20, "Career theme": "Advanced AI/research", "Risk note": "Often
        needs graduate-level preparation and strong math/research skills."},
    {"Occupation": "Software Developers / QA / Testers", "Projected growth %":
        15, "Career theme": "Software engineering", "Risk note": "Good outlook,

```

```

    but entry-level market can be competitive; projects matter."},
{"Occupation": "Computer Systems Analysts", "Projected growth %": 9, "Career
  theme": "Business + technology", "Risk note": "Communication and domain
  knowledge are important differentiators."},
{"Occupation": "Computer Hardware Engineers", "Projected growth %": 7,
  "Career theme": "Hardware/ECE", "Risk note": "Smaller labor market;
  internships and specialized skills matter."},
{"Occupation": "Computer Programmers", "Projected growth %": -6, "Career
  theme": "Routine coding pressure", "Risk note": "Routine coding alone is
  vulnerable; move toward design, systems, AI, security, or domain
  expertise."},
])

display(career_outlook)

fig = px.bar(
    career_outlook.sort_values("Projected growth %"),
    x="Projected growth %",
    y="Occupation",
    color="Career theme",
    orientation="h",
    title="Career Outlook Snapshot: Growth Is Uneven Across Tech Careers",
    text="Projected growth %",
    hover_data=["Risk note"],
)
fig.add_vline(x=0, line_dash="dash")
fig.update_layout(height=650, yaxis_title="")
fig.show()

```

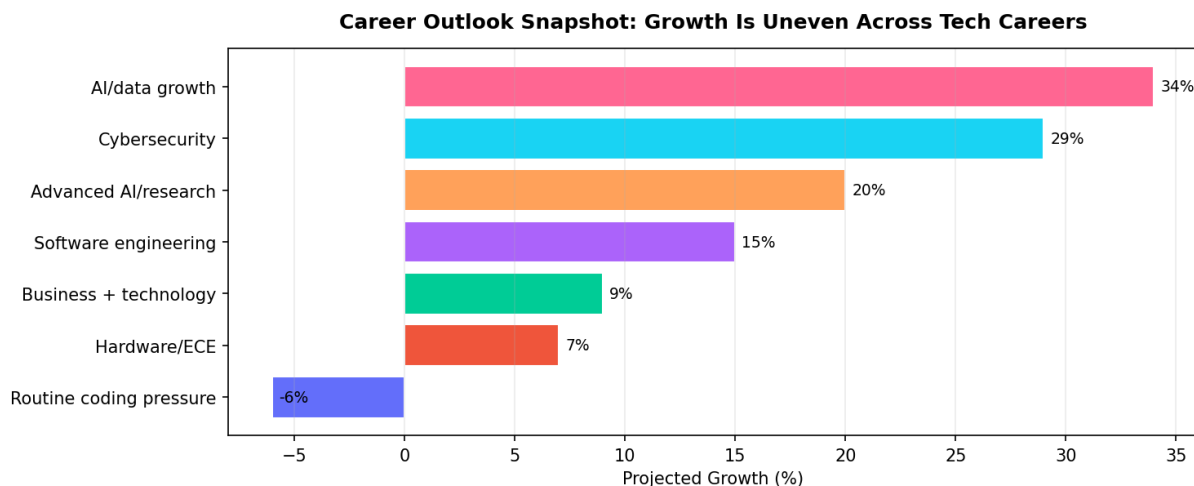


Figure 8: Career Outlook Snapshot: Growth Is Uneven Across Tech Careers

16. Choose a better 2D career comparison chart

The previous 3D career scatterplot has been removed because it was visually confusing and did not teach interpretation well. For this assignment, students should choose from clearer 2D chart options.

This section creates a small metrics table and then lets students select one chart type by changing

CAREER_CHART_CHOICE.

Available choices:

- "ranked_bar" — best for comparing total percent change.
- "risk_return_scatter" — best for comparing opportunity versus volatility.
- "bubble" — similar to scatter, but point size shows the latest index level.
- "heatmap" — best for seeing the whole timeline.
- "small_multiples" — best for avoiding the spaghetti-plot problem.

Pedagogical rule: students must explain **why** they chose the chart, not merely display it.

```
# Build a clean metrics table from the globally aligned window.
# This replaces the old 3D graph with clearer 2D choices.
matrix_slice =
    jobs_norm_wide.loc[JOB_COMMON_START:JOB_COMMON_END].dropna(how="any")

first = matrix_slice.iloc[0]
latest = matrix_slice.iloc[-1]
pct = (latest / first - 1) * 100
vol = matrix_slice.pct_change(fill_method=None).std() * 100

career_metrics = pd.DataFrame({
    "Career category": matrix_slice.columns,
    "Latest normalized index": latest.values,
    "Percent change since aligned start": pct.values,
    "Volatility": vol.values,
}).dropna().sort_values("Percent change since aligned start", ascending=False)

display(career_metrics)

# Students: choose one chart type from the list below.
# CAREER_CHART_CHOICE = "ranked_bar"
# CAREER_CHART_CHOICE = "risk_return_scatter"
CAREER_CHART_CHOICE = "bubble"
# CAREER_CHART_CHOICE = "heatmap"
# CAREER_CHART_CHOICE = "small_multiples"

if CAREER_CHART_CHOICE == "ranked_bar":
    fig = px.bar(
        career_metrics.sort_values("Percent change since aligned start"),
        x="Percent change since aligned start",
        y="Career category",
        orientation="h",
        text="Percent change since aligned start",
        title=f"Ranked Career Change: {JOB_COMMON_START.date()} to {JOB_COMMON_END.date()}",
    )
    fig.add_vline(x=0, line_dash="dash")
    fig.update_traces(texttemplate="%{text:.1f}%", textposition="outside")
    fig.update_layout(height=550, yaxis_title="")
    fig.show()

elif CAREER_CHART_CHOICE == "risk_return_scatter":
    fig = px.scatter(
        career_metrics,
        x="Percent change since aligned start",
```

```

y="Volatility",
color="Career category",
hover_name="Career category",
text="Career category",
title="Opportunity vs. Volatility: Clear 2D Scatter Plot",
)
fig.add_vline(x=0, line_dash="dash")
# ... (truncated for brevity)

```

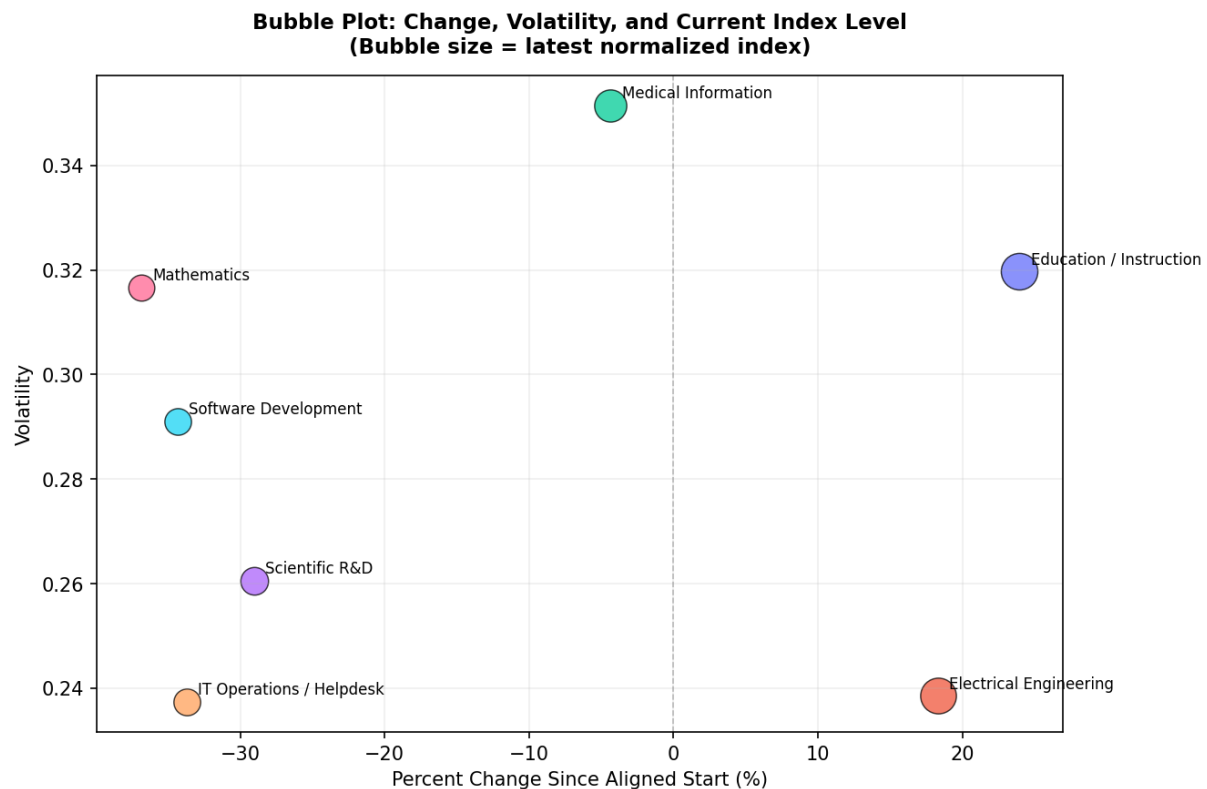


Figure 9: Bubble Plot: Change, Volatility, and Current Index Level

17. Word cloud — skills and career strategy

A word cloud is not a quantitative labor-market model, but it is a good communication visual. Students can edit the text to represent the skills they believe matter most.

```

try:
    from wordcloud import WordCloud

    skill_text = """
    Python Java SQL data structures algorithms cybersecurity cloud AI machine
    learning statistics
    communication teamwork internship portfolio Git GitHub Linux databases
    networking APIs visualization
    debugging testing documentation ethics mathematics probability calculus
    projects certification research
    presentation writing problem solving domain knowledge software engineering
    """

```

```

        systems design
    """

    wc = WordCloud(width=1200, height=500, background_color="white",
                    collocations=False).generate(skill_text)
    plt.figure(figsize=(14, 6))
    plt.imshow(wc, interpolation="bilinear")
    plt.axis("off")
    plt.title("Skills That Make Students More Competitive", fontsize=18)
    plt.show()
except Exception as e:
    print("WordCloud is not installed. In Colab run: !pip -q install wordcloud")
    print(e)

```

18. Student mini-report generator

Run this after the charts. It prints a structured paragraph template students can complete. This helps them connect visual evidence to career planning.

```

def latest_metric_text(change_df):
    strongest = change_df.sort_values("Percent change", ascending=False).iloc[0]
    weakest = change_df.sort_values("Percent change", ascending=True).iloc[0]
    return strongest, weakest

strongest, weakest = latest_metric_text(change_df)

report = f"""
### Draft Career Reflection

Using the aligned FRED/Indeed job-posting window from
    #{JOB_COMMON_START.date()}** to #{JOB_COMMON_END.date()}**, the career
    category with the strongest relative increase in this notebook was
    #{strongest['Career category']}** with approximately #{strongest['Percent
    change']:.1f}%** change. The weakest relative movement was #{weakest['Career
    category']}** with approximately #{weakest['Percent change']:.1f}%** change.

This suggests that students should not treat all technology careers as identical.
    A good career plan should include:

    1. a target role,
    2. evidence of skills through projects or internships,
    3. awareness of market volatility,
    4. continuous learning in AI, cybersecurity, data, cloud, or another
       specialization,
    5. communication skills that help explain technical work to non-technical people.

My selected career is: -----.

The evidence from at least two charts that supports my decision is:
    -----.

One risk in this career path is: -----.

One action I can take this semester to become more competitive is:
    -----.
    """

```

```
display(Markdown(report))
```

19. Optional extension for extra credit

Choose one extension:

- Add two more FRED career series and repeat all charts.
- Add two more states to the job-loser analysis.
- Replace the small manually entered outlook table with a verified current table from BLS.
- Use `CAREER_CHART_CHOICE` to create one additional clear 2D chart and explain why that chart type fits the question.
- Write a 1-page career memo explaining your target job, evidence from the notebook, and your skill-building plan.
- Compare one technical career with one non-technical career and explain whether the data supports your assumptions.

How to find more data

- **FRED:** First use the verified `OPTIONAL_JOB_SERIES_TO_TRY` menu above. Then, for careers not listed, search for **Indeed Hiring Lab** plus the occupation name, copy the FRED series ID, and add it to `JOB_SERIES`.
- **BLS Occupational Outlook Handbook:** Use this for salary, education requirement, projected growth, and work environment.
- **O\{*NET Online:** Use this for skills, tasks, tools, and abilities. This is useful for explaining *why* a career is growing or shrinking.
- **State labor departments:** Use these for local/regional career evidence.

Grading suggestion

Component	Points	Criteria	Points
Notebook runs without errors	20	At least four correct visualizations	25
Added one new career or state series	15	Correct interpretation of charts	25
Career reflection with action plan	15		

20. Optional dashboard cell — four plots in one figure

This final cell gives you a compact dashboard for screenshots or PDF export.

```
# Prepare dashboard data
norm_wide = jobs_norm_wide
latest_change = change_df.sort_values("Percent change")

fig = make_subplots(
    rows=2,
    cols=2,
    subplot_titles=(
        "Normalized trends, start=100",
        "Raw values as diagnostic view",
        "Percent change since aligned start",
        "New Jersey job losers",
    ),
)
```

```

    specs=[[{"type": "xy"}, {"type": "xy"}], [{"type": "xy"}, {"type": "xy"}]],
    horizontal_spacing=0.12,
    vertical_spacing=0.16,
)

for col in jobs_wide.columns:
    fig.add_trace(go.Scatter(x=norm_wide.index, y=norm_wide[col], mode="lines",
                             name=col, legendgroup=col), row=1, col=1)
    fig.add_trace(go.Scatter(x=jobs_wide.index, y=jobs_wide[col], mode="lines",
                             name=col, legendgroup=col, showlegend=False), row=1, col=2)

fig.add_trace(
    go.Bar(x=latest_change["Percent change"], y=latest_change["Career category"],
           orientation="h", name="% change", showlegend=False),
    row=2,
    col=1,
)

if "New Jersey" in state_losers_wide.columns:
    fig.add_trace(
        go.Scatter(x=state_losers_wide.index, y=state_losers_wide["New Jersey"],
                   mode="lines", name="NJ job losers", showlegend=False),
        row=2,
        col=2,
    )

fig.update_layout(
    title="Career Market Dashboard: Cleaned Comparisons, Change, and Pressure",
    height=900,
    hovermode="x unified",
)

fig.update_xaxes(title_text="Date", row=1, col=1)
fig.update_xaxes(title_text="Date", row=1, col=2)
fig.update_xaxes(title_text="Percent change", row=2, col=1)
fig.update_xaxes(title_text="Date", row=2, col=2)
fig.update_yaxes(title_text="Index, start=100", row=1, col=1)
fig.update_yaxes(title_text="Raw index", row=1, col=2)
fig.update_yaxes(title_text="Career", row=2, col=1)
fig.update_yaxes(title_text="People", row=2, col=2)
fig.show()

```

21. Matplotlib visualization skills lab

The earlier sections used mostly Plotly because interactive charts help with exploration. This lab teaches the lower-level Matplotlib concepts that students need for exams, reports, and debugging visual design decisions.

21A. Line plots: axes, labels, legends, colors, line styles, markers, grid, and fill areas

This cell demonstrates core Matplotlib syntax. It also shows conditional filling: values above 100 indicate the career index is above its aligned starting level; values below 100 indicate it is below that level.

```

focus_career = "Software Development"
comparison_career = "Electrical Engineering"

```

```

for career in [focus_career, comparison_career]:
    if career not in jobs_norm_wide.columns:
        raise KeyError(f"{career} is not available. Choose from
                        {list(jobs_norm_wide.columns)}")

fig, ax = plt.subplots(figsize=(12, 5))

ax.plot(
    jobs_norm_wide.index,
    jobs_norm_wide[focus_career],
    label=focus_career,
    linewidth=2.5,
    linestyle="-",
    marker="",
)
ax.plot(
    jobs_norm_wide.index,
    jobs_norm_wide[comparison_career],
    label=comparison_career,
    linewidth=2,
    linestyle="--",
    marker="o",
    markevery=max(1, len(jobs_norm_wide) // 12),
    markersize=4,
)

baseline = 100
focus_values = jobs_norm_wide[focus_career].astype(float)
ax.fill_between(
    jobs_norm_wide.index,
    focus_values,
    baseline,
    where=focus_values >= baseline,
    alpha=0.18,
    interpolate=True,
    label=f"{focus_career} above start",
)
ax.fill_between(
    jobs_norm_wide.index,
    focus_values,
    baseline,
    where=focus_values < baseline,
    alpha=0.18,
    interpolate=True,
    label=f"{focus_career} below start",
)

ax.axhline(baseline, linestyle=":", linewidth=2, label="Starting level = 100")
# ... (truncated for brevity)

```

21B. Histograms, bar charts, pie charts, overlays, and box plots

A **histogram** shows the distribution of one numeric variable. A **bar chart** compares categories. A **box plot** summarizes median, spread, and outliers. Pie charts are included for completeness, but for serious comparison, bar charts are usually clearer.

```

import matplotlib.patheffects as pe

returns_percent =
    jobs_norm_wide.pct_change(fill_method=None).mul(100).dropna(how="any")

# Histogram with edge color, median line, and optional log-scaled y-axis.
fig, ax = plt.subplots(figsize=(10, 4))
ax.hist(returns_percent[focus_career], bins=30, edgecolor="black", linewidth=0.8)
median_return = returns_percent[focus_career].median()
ax.axvline(median_return, linestyle="--", linewidth=2, label=f"Median =
    {median_return:.3f}%")
ax.set_title(f"Histogram of Daily Percent Changes: {focus_career}")
ax.set_xlabel("Daily percent change")
ax.set_ylabel("Frequency")
ax.set_yscale("linear") # Try "log" when a few bins dominate the chart.
ax.legend(loc="best")
ax.grid(True, alpha=0.25)
plt.tight_layout()
plt.show()

# Bar chart with edge color and shadow-like path effects.
bar_data = change_df.sort_values("Percent change", ascending=False).copy()
fig, ax = plt.subplots(figsize=(10, 5))
bars = ax.bar(bar_data["Career category"], bar_data["Percent change"],
    edgecolor="black", linewidth=1.0)
for bar in bars:
    bar.set_path_effects([pe.SimplePatchShadow(offset=(1.5, -1.5), alpha=0.25),
        pe.Normal())])
ax.axhline(0, linestyle="--", linewidth=1)
ax.set_title("Bar Chart: Percent Change by Career")
ax.set_ylabel("Percent change")
ax.set_xlabel("Career category")
plt.xticks(rotation=35, ha="right")
plt.tight_layout()
plt.show()

# Overlaid bar chart: first vs. latest normalized values.
metric_slice =
    jobs_norm_wide.loc[JOB_COMMON_START:JOB_COMMON_END].dropna(how="any")
first_latest = pd.DataFrame({"First": metric_slice.iloc[0], "Latest":
    metric_slice.iloc[-1]})
x = np.arange(len(first_latest.index))
width = 0.38
fig, ax = plt.subplots(figsize=(10, 5))
ax.bar(x - width / 2, first_latest["First"], width, label="First aligned value",
    edgecolor="black")
ax.bar(x + width / 2, first_latest["Latest"], width, label="Latest aligned
    value", edgecolor="black")
ax.set_title("Overlaid Bar Plot: First vs. Latest Normalized Values")
ax.set_xticks(x)
ax.set_xticklabels(first_latest.index, rotation=35, ha="right")
ax.set_ylabel("Normalized index")
ax.legend(loc="best")
ax.grid(True, axis="y", alpha=0.25)
plt.tight_layout()
plt.show()

```

```
# Box-and-whisker plot across careers.
# ... (truncated for brevity)
```

21C. Stack plots and stem plots

A **stack plot** is useful when parts contribute to a whole. A **stem plot** emphasizes discrete jumps, such as monthly percentage changes.

```
monthly_norm = jobs_norm_wide.resample("MS").mean().dropna(how="any")
career_share = monthly_norm.div(monthly_norm.sum(axis=1), axis=0).mul(100)

fig, ax = plt.subplots(figsize=(12, 5))
ax.stackplot(
    career_share.index,
    [career_share[col] for col in career_share.columns],
    labels=career_share.columns,
    alpha=0.85,
)
ax.set_title("Stack Plot with Constant Total: Career Share of Combined Normalized
    Index")
ax.set_xlabel("Date")
ax.set_ylabel("Percent of total normalized index")
ax.legend(loc="upper left", bbox_to_anchor=(1.02, 1.0))
ax.grid(True, alpha=0.2)
plt.tight_layout()
plt.show()

monthly_change =
    monthly_norm[focus_career].pct_change(fill_method=None).mul(100).dropna()
fig, ax = plt.subplots(figsize=(12, 4))
markerline, stemlines, baseline = ax.stem(monthly_change.index,
    monthly_change.values)
plt.setp(markerline, markersize=4)
plt.setp(stemlines, linewidth=1)
ax.axhline(0, linestyle="--", linewidth=1)
ax.set_title(f"Stem Plot: Monthly Percent Change for {focus_career}")
ax.set_xlabel("Date")
ax.set_ylabel("Monthly percent change")
ax.grid(True, alpha=0.25)
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

21D. Scatter plots: size, color, and marker edges

Scatter plots are useful for comparing two numeric variables. Here, each point is one career. The dot size represents the latest normalized index.

```
scatter_df = career_metrics.copy()

fig, ax = plt.subplots(figsize=(9, 6))
scatter = ax.scatter(
    scatter_df["Percent change since aligned start"],
    scatter_df["Volatility"],
    s=scatter_df["Latest normalized index"].clip(lower=1) * 2,
```

```

    c=np.arange(len(scatter_df)),
    edgecolors="black",
    linewidths=0.8,
    alpha=0.8,
)
for _, row in scatter_df.iterrows():
    ax.annotate(
        row["Career category"],
        (row["Percent change since aligned start"], row["Volatility"]),
        textcoords="offset points",
        xytext=(5, 4),
        fontsize=8,
    )
ax.set_title("Scatter Plot: Career Change vs. Volatility")
ax.set_xlabel("Percent change since aligned start")
ax.set_ylabel("Volatility")
ax.grid(True, alpha=0.25)
plt.tight_layout()
plt.show()

```

21E. Time-series dates: datetime conversion and connected data points

A time-series plot works correctly only when dates are actual datetime values. The FRED loader already performs this conversion, but students should know how to check it.

```

date_demo = jobs_long[["date", "name", "value"]].head(10).copy()
print("Before forced string conversion:")
print(date_demo.dtypes)

# Simulate a common beginner problem: dates arrive as strings.
date_demo["date_as_string"] = date_demo["date"].astype(str)
print("\nAfter creating a string date column:")
print(date_demo[["date_as_string"]].dtypes)

# Correct fix.
date_demo["date_fixed"] = pd.to_datetime(date_demo["date_as_string"],
    errors="coerce")
print("\nAfter pd.to_datetime:")
print(date_demo[["date_fixed"]].dtypes)

display(date_demo.head())

# Connected data points by line.
small = jobs_norm[jobs_norm["Career category"] == focus_career].head(50)
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(small["date"], small["Normalized index"], marker="o", linewidth=1)
ax.set_title("Connected Time-Series Points")
ax.set_xlabel("Date")
ax.set_ylabel("Normalized index")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()

# Live plotting note:
# Matplotlib FuncAnimation is useful for streaming data, but this assignment uses
  static public datasets.

```

```
# In a later project, students can animate one new observation at a time using
matplotlib.animation.FuncAnimation.
```

21F. Multiple subplots, separate figures, and controlled saving

This cell shows a 2-by-2 Matplotlib grid and demonstrates safe saving controlled by `SAVE_LOCAL_ARTIFACTS`. Students should not write files automatically unless the assignment explicitly requires it.

```
fig, axes = plt.subplots(2, 2, figsize=(14, 9))
axes = axes.ravel()

jobs_norm_wide[[focus_career, comparison_career]].plot(ax=axes[0])
axes[0].set_title("Line plot")
axes[0].set_ylabel("Normalized index")
axes[0].grid(True, alpha=0.25)

axes[1].hist(returns_percent[focus_career], bins=25, edgecolor="black")
axes[1].set_title("Histogram")
axes[1].set_xlabel("Daily percent change")

axes[2].bar(change_df["Career category"], change_df["Percent change"],
            edgecolor="black")
axes[2].set_title("Bar chart")
axes[2].tick_params(axis="x", rotation=35)

axes[3].scatter(scatter_df["Percent change since aligned start"],
                scatter_df["Volatility"], edgecolors="black")
axes[3].set_title("Scatter plot")
axes[3].set_xlabel("Percent change")
axes[3].set_ylabel("Volatility")
axes[3].grid(True, alpha=0.25)

plt.suptitle("Four Matplotlib Plots in One Figure", fontsize=14)
plt.tight_layout()

if SAVE_LOCAL_ARTIFACTS:
    fig.savefig(ARTIFACT_DIR / "matplotlib_four_panel_dashboard.png", dpi=200,
                bbox_inches="tight")
    print("Saved figure to", ARTIFACT_DIR / "matplotlib_four_panel_dashboard.png")

plt.show()

# Separate figures are better when each chart needs detailed interpretation.
fig, ax = plt.subplots(figsize=(8, 4))
jobs_norm_wide[focus_career].plot(ax=ax)
ax.set_title(f"Separate Figure: {focus_career}")
ax.set_ylabel("Normalized index")
ax.grid(True, alpha=0.25)
plt.tight_layout()
plt.show()
```

22. Seaborn visualization skills lab

Seaborn builds on Matplotlib and is useful for statistical plotting. This section demonstrates hue, style, size, line plots, categorical plots, pair plots, joint/regression-style plots, subplots, and aesthetics.

```
try:
    import seaborn as sns

    sns.set_theme(style="whitegrid", context="notebook")

    # Reduce the line plot to monthly data so the Seaborn example stays readable.
    seaborn_monthly = (
        jobs_norm_wide
        .resample("MS")
        .mean()
        .reset_index()
        .melt(id_vars="date", var_name="Career category", value_name="Normalized
            index")
        .dropna()
    )

    plt.figure(figsize=(12, 5))
    sns.lineplot(
        data=seaborn_monthly,
        x="date",
        y="Normalized index",
        hue="Career category",
        style="Career category",
        markers=False,
    )
    plt.title("Seaborn Line Plot with Hue and Style")
    plt.xticks(rotation=30)
    plt.tight_layout()
    plt.show()

    # Cat plot / ranked bar chart.
    cat = sns.catplot(
        data=change_df,
        x="Percent change",
        y="Career category",
        kind="bar",
        height=4.5,
        aspect=1.8,
    )
    cat.fig.suptitle("Seaborn Cat Plot: Percent Change by Career", y=1.03)
    plt.show()

    # Pair plot for multiple numeric metrics.
    sns.pairplot(
        career_metrics,
        vars=["Latest normalized index", "Percent change since aligned start",
            "Volatility"],
        hue="Career category",
        corner=True,
    )
    plt.suptitle("Seaborn Pair Plot: Career Metrics", y=1.02)
```

```
plt.show()
# ... (truncated for brevity)
```

23. Plotly and optional Cufflinks lab

Plotly is used for interactive charts. Cufflinks connects Pandas dataframes to Plotly-style `.iplot()` calls. Cufflinks is optional; the notebook remains valid if it is not installed.

```
# Plotly line, scatter/bubble, bar, box, area, histogram, and heatmap examples.
fig = px.area(
    jobs_norm,
    x="date",
    y="Normalized index",
    color="Career category",
    title="Plotly Area Plot: Normalized Career Indices",
)
fig.update_layout(height=500)
fig.show()

fig = px.box(
    returns_percent.reset_index().melt(id_vars="date", var_name="Career
        category", value_name="Daily percent change"),
    x="Career category",
    y="Daily percent change",
    title="Plotly Box Plot: Daily Percent Changes",
)
fig.update_layout(height=500)
fig.show()

fig = px.histogram(
    returns_percent.reset_index().melt(id_vars="date", var_name="Career
        category", value_name="Daily percent change"),
    x="Daily percent change",
    color="Career category",
    marginal="box",
    title="Plotly Histogram with Marginal Box Plot",
)
fig.update_layout(height=500)
fig.show()

fig = px.scatter(
    career_metrics,
    x="Percent change since aligned start",
    y="Volatility",
    size="Latest normalized index",
    color="Career category",
    hover_name="Career category",
    title="Plotly Bubble Plot: Change, Volatility, and Latest Index",
)
fig.update_layout(height=550)
fig.show()

fig = px.imshow(
    jobs_norm_wide.T,
    aspect="auto",
    title="Plotly Heatmap: Career Intensity Over Time",
```

```

    labels=dict(x="Date", y="Career category", color="Index"),
)
fig.update_layout(height=500)
fig.show()
# ... (truncated for brevity)

```

24. Student extension checklist

Before submitting, students should complete these steps:

- Add at least one new profession to `JOB_SERIES`.
- Re-run the whole notebook from the top.
- Check whether the new profession changes `JOB_COMMON_START` or `JOB_COMMON_END`.
- Explain whether timeline truncation helped or harmed the analysis.
- Choose the three most informative visualizations and justify why.
- Include at least one Matplotlib chart, one Seaborn chart, and one Plotly chart in the final write-up.
- Cite any external source used beyond FRED.

A strong submission should not merely show charts. It should explain what each chart reveals, what it hides, and whether the evidence is strong enough to support a career-planning conclusion.

```

#The bar chart is the best choice for comparing careers.
#A bar chart shows one summary value per career on the same numeric scale,
#so it is easy to see which careers have higher or lower values at a glance by
#comparing bar lengths. Histograms and box plots focus on the distribution of many
#observations for each variable, which is more useful for understanding
variability than
#for simple side by side category comparison. Pie charts are harder to read
accurately
#when there are many categories and make it difficult to compare small
differences,
#so they are less effective than a bar chart for comparing multiple careers.

```

```

#4
#When discussing trends or relative performance,
#knowing that all data points correspond to the same time period makes the
analysis
#much clearer and more robust. While it might slightly reduce the overall data
available
#(by dropping observations outside the common window),
#this trade-off is generally beneficial for ensuring the validity and fairness
#of comparative analysis, especially
#with FRED data where series can have different start and end dates.

```

Student Response:

Visualization 1: "Relative Job-Market Change: Every Career Starts at 100" (Plotly Line Plot, Cell f8f98a6c):

This chart immediately shows the relative performance of each career against its own starting

point. By normalizing all series to 100 at the first common date, it allows for a direct comparison of growth or decline rates without being influenced by different absolute magnitudes of the raw index values. You can quickly see which careers are trending up, down, or sideways relative to their starting conditions.

Visualization 3: "Percent Change in Job-Posting Index Since [Start Date]" (Plotly Bar Chart, Cell 9d3949e4): This bar chart provides a clear, quantitative summary of each career's overall change over the entire analysis period. It's excellent for quickly identifying the top-performing and worst-performing careers in terms of total growth or decline. The visual ranking makes it easy to grasp the 'winners' and 'losers' in the job market during the given timeframe.

Visualization 11: "Estimated Career-Market Volatility Based on Normalized Job-Posting Changes" (Plotly Bar Chart, Cell d4465e83):

While growth is important, stability is also a key factor in career planning. This chart quantifies the volatility of each career, showing which ones experience more frequent and larger fluctuations in job postings. This is crucial for students to understand the risk associated with a career path; a high-growth career might also be highly volatile, suggesting a less predictable path, while a stable career might offer more security even with moderate growth. It provides a different dimension of analysis beyond just growth.